SIMATS ENGINEERING
ASSESSMENT TOOL 3 - LOW (ASSIGNMENT - 2)

COURSE CODE: CSA09 **COURSE NAME: Programming in Java**

COURSE OUTCOME COVERED

CO3: *Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques (BL3,BL4)*

Assessment Tool Weightage: 10%

QUESTIONS: 1 **Total Marks: 50**

ANNEXURE A

ASSIGNMENT - 2

Code of Conduct:

I _____SIVARANGJENII B(192521375)_____ *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
SIVARANGJENII B

ANNEXURE

ASSIGNMENT – 2

1: Banking Transaction and Exception Management System

A nationalized bank is developing a Java-based banking application to manage customer accounts and financial transactions. The system should accept account details, deposits, withdrawals, and fund transfers. Since financial applications must handle invalid user inputs and unexpected runtime conditions safely, the application should not terminate abruptly when an error occurs. Invalid account numbers, insufficient balances, negative transaction amounts, invalid numeric inputs, and attempts to access unavailable accounts must be handled appropriately. The system should also maintain transaction processing using separate threads so that multiple customers can perform transactions concurrently.

Develop a Java application that performs the following:

a) Create suitable classes for Customer, Account, and Transaction.

b) Implement deposit, withdrawal, and fund-transfer operations using appropriate **exception handling mechanisms**.

c) Handle built-in exceptions such as `ArithmeticException`, `NumberFormatException`, `InputMismatchException`, and `NullPointerException` wherever applicable.

d) Implement multiple `try-catch` blocks and demonstrate the use of the **finally** block for closing or releasing resources.

e) Create a user-defined exception named `InsufficientBalanceException` and generate it whenever a withdrawal exceeds the available balance.

f) Create separate threads for processing multiple customer transactions simultaneously.

g) Use **synchronization** to ensure that two threads cannot modify the same account balance simultaneously.

h) Demonstrate an appropriate mechanism for handling an **uncaught exception** at the thread level.

i) Display the final account balance and transaction status after all threads complete execution.

1. RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	15%	Clearly understands the problem and requirements	Understands most requirements	Understands basic requirements	Has difficulty understanding requirements
Application of Java Concepts	20%	Applies relevant Java concepts correctly and effectively	Applies concepts with minor errors	Applies concepts with some limitations	Applies concepts incorrectly
Program Design & Logic	15%	Logical, modular, and efficient design	Well-structured design with minor issues	Basic design with limited logic	Poor or unclear design
Program Functionality	20%	Fully functional and produces correct results	Mostly functional with minor errors	Partially functional with some errors	Incomplete or non-functional
Error Handling	10%	Handles errors and invalid inputs effectively	Handles major errors appropriately	Handles limited error conditions	Provides little or no error handling
Testing & Validation	10%	Tests multiple cases and validates results	Tests major cases adequately	Performs limited testing	Performs minimal or no testing
Code Quality & Documentation	5%	Clean, readable, and well documented	Good code with adequate documentation	Basic code and documentation	Poorly organized and documented
Output & Presentation	5%	Clear output with effective analysis and presentation	Good output and presentation	Basic output and presentation	Unclear or incomplete output

CODE:

```
class Main {
    public static void main(String[] args) {

        int id1 = 101;
        String name1 = "Arun";
        int no1 = 1001;
        double bal1 = 10000;

        int id2 = 102;
        String name2 = "Bala";
        int no2 = 1002;
        double bal2 = 5000;

        String type1 = "withdraw";
        double amount1 = 2000;

        String type2 = "deposit";
        double amount2 = 3000;

        Account a1 = new Account(no1, bal1);
        Account a2 = new Account(no2, bal2);
```

```
    Customer c1 = new Customer(id1, name1, a1);
    Customer c2 = new Customer(id2, name2, a2);

    Transaction tr1 = new Transaction(c1, type1, amount1, null);
    Transaction tr2 = new Transaction(c2, type2, amount2, null);

    Thread t1 = new Thread(tr1, "Customer1");
    Thread t2 = new Thread(tr2, "Customer2");

    t1.start();
    t2.start();

    try {
        t1.join();
        t2.join();
    } catch (InterruptedException e) {
        System.out.println("Thread interrupted");
    }

    System.out.println("Customer 1 Status: " + tr1.status);
    System.out.println("Customer 1 Balance: " + a1.balance);
    System.out.println("Customer 2 Status: " + tr2.status);
    System.out.println("Customer 2 Balance: " + a2.balance);
    System.out.println("Application completed");
}
```

```
}

class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String msg) {
        super(msg);
    }
}

class Customer {
    int id;
    String name;
    Account account;

    Customer(int id, String name, Account account) {
        this.id = id;
        this.name = name;
        this.account = account;
    }
}

class Account {
    int accNo;
    double balance;

    Account(int accNo, double balance) {
```

```
        this.accNo = accNo;
        this.balance = balance;
    }

    synchronized void deposit(double amount) {
        if (amount < 0)
            throw new IllegalArgumentException("Negative amount");
        balance += amount;
    }

    synchronized void withdraw(double amount)
            throws InsufficientBalanceException {
        if (amount < 0)
            throw new IllegalArgumentException("Negative amount");

        if (amount > balance)
            throw new InsufficientBalanceException("Insufficient
balance");

        balance -= amount;
    }

    synchronized void transfer(Account a, double amount)
            throws InsufficientBalanceException {
        if (a == null)
```

```
            throw new NullPointerException("Account not found");

        if (amount < 0)
            throw new IllegalArgumentException("Negative amount");

        if (amount > balance)
            throw new InsufficientBalanceException("Insufficient
balance");

        balance -= amount;
        a.balance += amount;
    }
}

class Transaction implements Runnable {
    Customer c;
    String type;
    double amount;
    Account receiver;
    String status;

    Transaction(Customer c, String type, double amount, Account
receiver) {
        this.c = c;
        this.type = type;
```

```
        this.amount = amount;
        this.receiver = receiver;
    }

    public void run() {
        try {
            if (type.equalsIgnoreCase("deposit"))
                c.account.deposit(amount);
            else if (type.equalsIgnoreCase("withdraw"))
                c.account.withdraw(amount);
            else if (type.equalsIgnoreCase("transfer"))
                c.account.transfer(receiver, amount);
            else
                throw new IllegalArgumentException("Invalid
transaction");

            status = "SUCCESS";

        } catch (InsufficientBalanceException e) {
            status = e.getMessage();
        } catch (Exception e) {
            status = e.getMessage();
        }
```

```
        System.out.println(Thread.currentThread().getName() + "
completed");
    }
}
```

OUTPUT: